a3

April 15, 2026

[1]:
```
%pip install -q lightgbm catboost scikit-learn pandas numpy
```

```
[notice] A new release of pip is
available: 26.0 -> 26.0.1
[notice] To update, run:
python -m pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
```

[2]:
```
import pandas as pd

TRAIN_DATA  = pd.read_csv('train-data.csv',  index_col='id')
TRAIN_LABEL = pd.read_csv('train-label.csv', index_col='id')
TEST_DATA   = pd.read_csv('test-data.csv',   index_col='id')
```

[3]:
```
import numpy as np
import pandas as pd

def preprocess(df):
    df = df.copy()

    drop_cols = [
        'first_name', 'last_name', 'insitute_name', 'institute_location',
        'test_1', 'test_2', 'test_3', 'test_4', 'test_5', 'treatment_consent'
    ]
    df = df.drop(columns=drop_cols)

    # Missingness features BEFORE encoding (class 9 has higher missing rate)
    miss_cols = [
        'gender', 'maternal_defect', 'mother_age', 'father_age',
        'respiration', 'heart_rate', 'risk_level', 'place_birth',
        'folic_acid', 'maternal_illness', 'infertility_treatment',
        'problem_previous_pregnancies', 'abortion_cnt',
        'birth_defects', 'white_blood_cell_count', 'blood_test',
        'symptom_1', 'symptom_2', 'symptom_3', 'symptom_4', 'symptom_5'
    ]
    df['missing_count']      = df[miss_cols].isna().sum(axis=1)
```

```
    df['missing_parent_age'] = df['mother_age'].isna().astype(int) +␣
↪df['father_age'].isna().astype(int)
    df['missing_symptoms']   =␣
↪df[['symptom_1','symptom_2','symptom_3','symptom_4','symptom_5']].isna().
↪sum(axis=1)
    df['missing_clinical']   =␣
↪df[['respiration','heart_rate','risk_level','blood_test']].isna().sum(axis=1)
    for col in ['mother_age','father_age','maternal_defect','gender',
                ␣
↪'risk_level','heart_rate','respiration','abortion_cnt','white_blood_cell_count']:
↪
        df[f'{col}_missing'] = df[col].isna().astype(int)

    # Encode
    binary_yn = [
        'mother_defect','father_defect','maternal_defect','paternal_defect',
        'alive','folic_acid','maternal_illness','infertility_treatment',
        'problem_previous_pregnancies',
        'symptom_1','symptom_2','symptom_3','symptom_4','symptom_5'
    ]
    for col in binary_yn:
        df[col] = df[col].map({'Y':1,'N':0})
    df['respiration']   = df['respiration'].map({'A':1,'N':0})
    df['heart_rate']    = df['heart_rate'].map({'A':1,'N':0})
    df['risk_level']    = df['risk_level'].map({'H':1,'L':0})
    df['place_birth']   = df['place_birth'].map({'I':1,'H':0})
    df['birth_defects'] = df['birth_defects'].map({'S':1,'M':2})
    df['gender']        = df['gender'].map({'M':0,'F':1,'A':2})
    df['autopsy']       = df['autopsy'].map({'Y':1,'N':0})
    for col in ['birth_asphyxia','radiation_exposure','substance_abuse']:
        df[col] = df[col].map({'Y':1,'N':0,'NR':2})
    df['blood_test'] = df['blood_test'].map({'N':0,'I':1,'S':2,'A':3})

    # Engineered features
    df['defect_sum']       =␣
↪df[['mother_defect','father_defect','maternal_defect','paternal_defect']].
↪sum(axis=1)
    df['symptom_sum']      =␣
↪df[['symptom_1','symptom_2','symptom_3','symptom_4','symptom_5']].sum(axis=1)
    df['defect_x_symptom'] = df['defect_sum'] * df['symptom_sum']
    df['any_defect']       = (df['defect_sum'] > 0).astype(int)
    df['any_symptom']      = (df['symptom_sum'] > 0).astype(int)
    df['high_symptom']     = (df['symptom_sum'] >= 4).astype(int)
    df['all_defects']      = (df['defect_sum'] == 4).astype(int)
    df['parent_age_gap']   = (df['father_age'] - df['mother_age']).abs()
    df['symptom_defect_ratio'] = df['symptom_sum'] / (df['defect_sum'] + 1)
```

```
    # Symptom pattern features (highly discriminative)
    df['s4_and_s5']   = ((df['symptom_4']==1) & (df['symptom_5']==1)).
 ↪astype(int)
    df['no_s4_s5']    = ((df['symptom_4']==0) & (df['symptom_5']==0)).
 ↪astype(int)
    df['late_vs_early'] = (df['symptom_4'].fillna(0) + df['symptom_5'].fillna(0)
                           - df['symptom_1'].fillna(0) - df['symptom_2'].
 ↪fillna(0))
    df['weighted_sym']  = (df['symptom_1'].fillna(0)*1 + df['symptom_2'].
 ↪fillna(0)*1 +
                           df['symptom_3'].fillna(0)*1 + df['symptom_4'].
 ↪fillna(0)*2 +
                           df['symptom_5'].fillna(0)*2)
    df['both_parents_defect'] = ((df['mother_defect']==1) &␣
 ↪(df['father_defect']==1)).astype(int)
    df['no_parent_defect']    = ((df['mother_defect']==0) &␣
 ↪(df['father_defect']==0)).astype(int)

    return df

X_train = preprocess(TRAIN_DATA)
X_test  = preprocess(TEST_DATA)
y_train = TRAIN_LABEL['disorder'].values

print("X_train shape:", X_train.shape)
print("X_test shape:",  X_test.shape)
```

```
X_train shape: (13249, 59)
X_test shape: (8834, 59)
```

[4]:
```
from catboost import CatBoostClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import balanced_accuracy_score
import numpy as np
import pandas as pd

N_SPLITS = 10
SEEDS    = [42, 7, 123]

# Class weights: inverse frequency (balanced)
class_counts  = np.bincount(y_train)
class_weights = len(y_train) / (10 * class_counts)

all_test_preds = np.zeros((len(X_test), 10))
all_oof_proba  = np.zeros((len(y_train), 10))
```

```
for SEED in SEEDS:
    print(f"\n{'='*40}  SEED={SEED}  {'='*40}")
    skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)

    oof_proba  = np.zeros((len(y_train), 10))
    test_preds = np.zeros((len(X_test), 10))
    fold_scores = []

    for fold, (tr_idx, val_idx) in enumerate(skf.split(X_train, y_train)):
        X_tr, X_val = X_train.iloc[tr_idx], X_train.iloc[val_idx]
        y_tr, y_val = y_train[tr_idx], y_train[val_idx]

        model = CatBoostClassifier(
            iterations          = 2000,
            learning_rate       = 0.03,
            depth               = 6,
            class_weights       = class_weights,
            l2_leaf_reg         = 3,
            random_seed         = SEED,
            verbose             = 0,
            thread_count        = -1,
            early_stopping_rounds = 100,
            eval_metric         = 'Accuracy',
        )
        model.fit(
            X_tr, y_tr,
            eval_set=(X_val, y_val),
            use_best_model=True
        )

        val_proba  = model.predict_proba(X_val)
        val_pred   = np.argmax(val_proba, axis=1)
        score      = balanced_accuracy_score(y_val, val_pred)
        fold_scores.append(score)
        print(f"  Fold {fold+1:2d}: BA={score:.4f}  best_iter={model.
↪best_iteration_}")

        oof_proba[val_idx] += val_proba
        test_preds         += model.predict_proba(X_test) / N_SPLITS

    oof_score = balanced_accuracy_score(y_train, np.argmax(oof_proba, axis=1))
    print(f"  OOF BA (seed={SEED}): {oof_score:.4f}  |  mean fold: {np.
↪mean(fold_scores):.4f} ± {np.std(fold_scores):.4f}")

    all_oof_proba  += oof_proba  / len(SEEDS)
    all_test_preds += test_preds / len(SEEDS)
```

```python
# Final scores
final_oof_score = balanced_accuracy_score(y_train, np.argmax(all_oof_proba,␣
↪axis=1))
print(f"\n{'='*90}")
print(f"FINAL OOF BA (averaged over {len(SEEDS)} seeds): {final_oof_score:.4f}")
print(f"{'='*90}")
```

```
========================================  SEED=42
========================================
  Fold  1: BA=0.3791  best_iter=146
  Fold  2: BA=0.4142  best_iter=31
  Fold  3: BA=0.3981  best_iter=26
  Fold  4: BA=0.3728  best_iter=282
  Fold  5: BA=0.3991  best_iter=2
  Fold  6: BA=0.4520  best_iter=137
  Fold  7: BA=0.3511  best_iter=30
  Fold  8: BA=0.3710  best_iter=45
  Fold  9: BA=0.4241  best_iter=55
  Fold 10: BA=0.4243  best_iter=51
  OOF BA (seed=42): 0.3984  |  mean fold: 0.3986 ± 0.0291

========================================  SEED=7
========================================
  Fold  1: BA=0.3853  best_iter=175
  Fold  2: BA=0.4051  best_iter=117
  Fold  3: BA=0.4169  best_iter=168
  Fold  4: BA=0.4145  best_iter=123
  Fold  5: BA=0.3705  best_iter=18
  Fold  6: BA=0.4045  best_iter=81
  Fold  7: BA=0.3728  best_iter=42
  Fold  8: BA=0.3782  best_iter=27
  Fold  9: BA=0.4131  best_iter=186
  Fold 10: BA=0.3779  best_iter=78
  OOF BA (seed=7): 0.3942  |  mean fold: 0.3939 ± 0.0177

========================================  SEED=123
========================================
  Fold  1: BA=0.4101  best_iter=220
  Fold  2: BA=0.3838  best_iter=56
  Fold  3: BA=0.3765  best_iter=214
  Fold  4: BA=0.3772  best_iter=103
  Fold  5: BA=0.4382  best_iter=96
  Fold  6: BA=0.3948  best_iter=111
  Fold  7: BA=0.3697  best_iter=173
  Fold  8: BA=0.3848  best_iter=114
  Fold  9: BA=0.4147  best_iter=29
  Fold 10: BA=0.3930  best_iter=97
```

```
  OOF BA (seed=123): 0.3945  |  mean fold: 0.3943 ± 0.0200

================================================================================
==========
FINAL OOF BA (averaged over 3 seeds): 0.3860
================================================================================
==========
```

[5]:
```python
final_preds = np.argmax(all_test_preds, axis=1)

submission = pd.DataFrame({
    'id':       TEST_DATA.index,
    'disorder': final_preds
}).set_index('id')

submission.to_csv('submission-v3.csv')
print("Saved! Shape:", submission.shape)
print(submission['disorder'].value_counts().sort_index())
```

```
Saved! Shape: (8834, 1)
disorder
0     382
1    1392
2     633
3    1562
4     248
5    1287
6    1081
7    1168
8     242
9     839
Name: count, dtype: int64
```